

JAVA LAB WEEK 4&5

EXPERIMENT : Farm Animals

1. Basic Information

Experiment No.: 01

Date: 17/09/2026

Title: Farm Animals Application using Classes, Objects, Constructors and Methods

2. Objective / Aim

To design and implement a Java program for a **Farm Animals application** using classes and objects to store and display the details of animals such as cow, pig, and horse.

The program demonstrates **classes, objects, constructors, constructor overloading, methods, method overloading, and parameter passing**.

3. System Requirements

Hardware Requirements

- Processor: Intel Core i3 or above
- RAM: Minimum 4 GB
- Hard Disk: Minimum 10 GB free space
- Keyboard and Monitor

Software Requirements

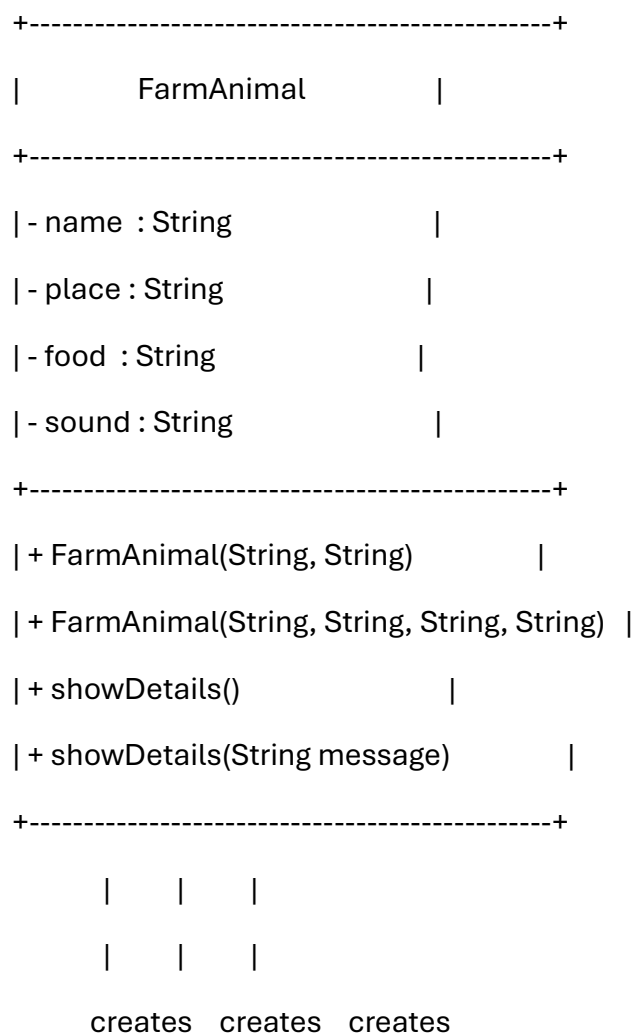
- Operating System: Windows/Linux/macOS
- JDK: JDK 8 or above
- IDE: IntelliJ IDEA, Eclipse, NetBeans, or Visual Studio Code
- Command Prompt/Terminal

4. Algorithm / Logic Description

1. Start the program.
2. Create a class named FarmAnimal.
3. Declare the attributes name, place, food, and sound.
4. Create a constructor with name and place as parameters.
5. Initialize food and sound with default values in the first constructor.
6. Create another constructor with name, place, food, and sound.

7. This demonstrates **constructor overloading**.
8. Create the showDetails() method to display the animal details.
9. Create another showDetails(String message) method.
10. This demonstrates **method overloading** and parameter passing.
11. Create objects cow, pig, and horse of the FarmAnimal class.
12. Initialize the objects with their respective names, places, food, and sounds.
13. Call showDetails() for the cow.
14. Call showDetails() for the pig.
15. Call showDetails(String message) for the horse.
16. Display the details of all three animals.
17. Stop the program.

5. UML Class Diagram



```

|      |      |
+-----+ +-----+ +-----+
| Cow | | Pig | | Horse |
+-----+ +-----+ +-----+

```

6. Source Code

```

class FarmAnimal {

    String name;

    String place;

    String food;

    String sound;

    FarmAnimal(String name, String place) {

        this.name = name;

        this.place = place;

        this.food = "Grass";

        this.sound = "Animal Sound";

    }

    FarmAnimal(String name, String place, String food, String sound) {

        this.name = name;

        this.place = place;

        this.food = food;

        this.sound = sound;

    }

    void showDetails() {

        System.out.println("Animal: " + name);

        System.out.println("Stays in: " + place);
    }
}

```

```

        System.out.println("Eats: " + food);
        System.out.println("Sound: " + sound);
    }

    void showDetails(String message) {
        System.out.println(message);
        showDetails();
    }
}

public class FarmAnimals {
    public static void main(String[] args) {

        FarmAnimal cow = new FarmAnimal(
            "Cow", "Cowshed", "Grass", "Moo"
        );

        FarmAnimal pig = new FarmAnimal(
            "Pig", "Pigsty", "Vegetables", "Oink"
        );

        FarmAnimal horse = new FarmAnimal(
            "Horse", "Stable", "Grass", "Neigh"
        );

        cow.showDetails();
        System.out.println();
    }
}

```

```
pig.showDetails();  
  
System.out.println();  
  
horse.showDetails("Horse Details:");  
}  
}
```

7. Compilation & Execution Commands

Java Filename: FarmAnimals.java

Compilation

```
javac FarmAnimals.java
```

Execution

```
java FarmAnimals
```

8. Output Screen

Animal: Cow

Stays in: Cowshed

Eats: Grass

Sound: Moo

Animal: Pig

Stays in: Pigsty

Eats: Vegetables

Sound: Oink

Horse Details:

Animal: Horse

Stays in: Stable

Eats: Grass

Sound: Neigh

9. Result

The **Farm Animals application** was successfully implemented and executed. The program demonstrates **classes, objects, constructors, constructor overloading, methods, method overloading, and parameter passing** in Java.

EXPERIMENT :Order Management

1. Basic Information

Experiment No.: 02

Date: 17/09/2026

Title: Order Management System using Inheritance and Method Overloading

2. Objective / Aim

To design and implement a simple **Order Management System in Java** using classes and objects to manage customers, normal orders, and special orders.

The program demonstrates **classes, objects, constructors, constructor overloading, inheritance, method overloading, method overriding, parameter passing, and the super keyword**.

3. System Requirements

Hardware Requirements

- Processor: Intel Core i3 or above
- RAM: Minimum 4 GB
- Hard Disk: Minimum 10 GB free space
- Keyboard and Monitor

Software Requirements

- Operating System: Windows/Linux/macOS
- JDK: JDK 8 or above
- IDE: IntelliJ IDEA, Eclipse, NetBeans, or Visual Studio Code

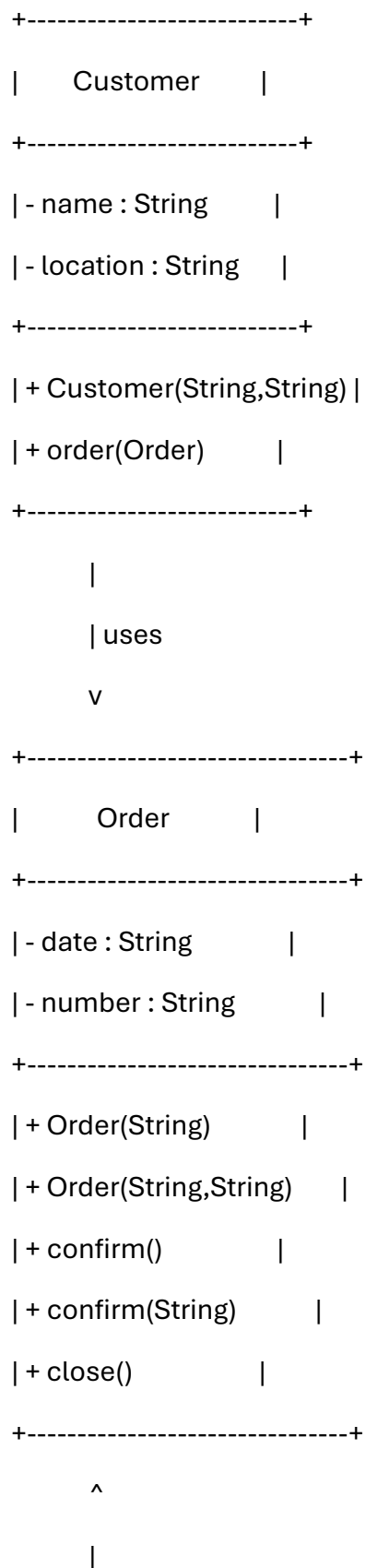
- Command Prompt/Terminal

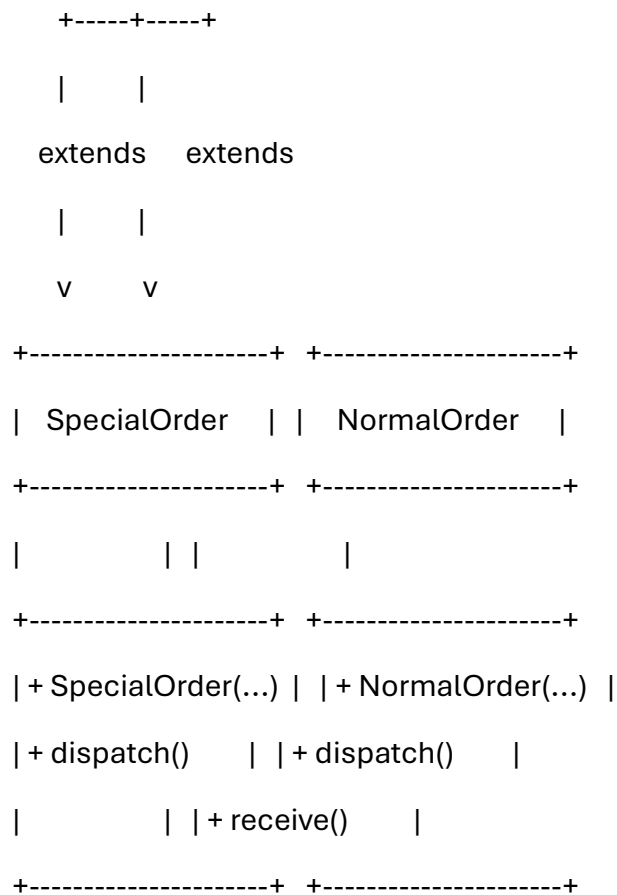
4. Algorithm / Logic Description

1. Start the program.
2. Create a class Customer with name and location attributes.
3. Create a constructor in Customer to initialize the customer details.
4. Define the order() method to accept an Order object.
5. Create a class Order with date and number attributes.
6. Create a constructor with only the order number.
7. Set the date to 17-09-2026 in this constructor.
8. Create another constructor with order number and date.
9. This demonstrates **constructor overloading**.
10. Define the confirm() method to confirm an order.
11. Define another confirm(String type) method.
12. This demonstrates **method overloading**.
13. Define the close() method to close an order.
14. Create SpecialOrder as a subclass of Order.
15. Use the super() constructor to initialize the order number and date.
16. Define the dispatch() method for a special order.
17. Create NormalOrder as another subclass of Order.
18. Define the dispatch() and receive() methods for a normal order.
19. Create a Customer object named c.
20. Create a SpecialOrder object named s.
21. Pass the special order to the customer's order() method.
22. Confirm, dispatch, and close the special order.
23. Create a NormalOrder object named n.
24. Pass the normal order to the customer's order() method.
25. Confirm, dispatch, receive, and close the normal order.
26. Display all results.

27. Stop the program.

5. UML Class Diagram





6. Source Code

```

class Customer {
    String name, location;

    Customer(String name, String location) {
        this.name = name;
        this.location = location;
    }

    void order(Order o) {
        System.out.println(name + " placed " + o.number);
    }
}

```

```
class Order {  
    String date, number;  
  
    Order(String number) {  
        this.number = number;  
        date = "17-09-2026";  
    }  
  
    Order(String number, String date) {  
        this.number = number;  
        this.date = date;  
    }  
  
    void confirm() {  
        System.out.println(number + " Confirmed");  
    }  
  
    void confirm(String type) {  
        System.out.println(type + " Order Confirmed");  
    }  
  
    void close() {  
        System.out.println(number + " Closed");  
    }  
}  
  
class SpecialOrder extends Order {  
    SpecialOrder(String n, String d) {
```

```

        super(n, d);
    }

    void dispatch() {
        System.out.println("Special Order Dispatched");
    }
}

class NormalOrder extends Order {
    NormalOrder(String n, String d) {
        super(n, d);
    }

    void dispatch() {
        System.out.println("Normal Order Dispatched");
    }

    void receive() {
        System.out.println("Normal Order Received");
    }
}

public class OrderManagement {
    public static void main(String[] args) {

        Customer c = new Customer("Ravi", "Vizag");

        SpecialOrder s = new SpecialOrder("S101", "17-09-2026");
    }
}

```

```
c.order(s);  
s.confirm("Special");  
s.dispatch();  
s.close();
```

```
NormalOrder n = new NormalOrder("N101", "17-09-2026");
```

```
c.order(n);  
n.confirm();  
n.dispatch();  
n.receive();  
n.close();
```

```
}
```

```
}
```

7. Compilation & Execution Commands

Java Filename: OrderManagement.java

Compilation

```
javac OrderManagement.java
```

Execution

```
java OrderManagement
```

8. Output Screen

Ravi placed S101

Special Order Confirmed

Special Order Dispatched

S101 Closed

Ravi placed N101

N101 Confirmed

Normal Order Dispatched

Normal Order Received

N101 Closed

9. Result

The **Order Management System** was successfully implemented and executed. The program demonstrates **classes, objects, constructors, constructor overloading, inheritance, method overloading, parameter passing, and the super keyword** in Java.

OVERALL RESULT

The given Java programs were successfully implemented and executed.

The experiments demonstrate the following Java/OOP concepts:

- Classes and Objects
- Constructors
- Constructor Overloading
- Methods
- Method Overloading
- Parameter Passing
- Inheritance
- extends keyword
- super keyword
- Console Output